

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	Total Marks:	1 * 100 = 100 (1 Question1)
CO2 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	Y. Jayesh Ranjan	Reg. No.:	192472201
Section / Batch:	CSE - AI	Date:	04/08/2026

Instructions to Students

- Answer 1 question. Total: 100 Marks.
- Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
- Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
- Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
- Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
- Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
- Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
- Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
- Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application.

Submission Requirements

Students should submit:

- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct

I Y. Jayesh Ranjan – 192472201 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y. Jayesh Ranjan

26. Sequential Function Calling

A travel assistant receives the request:

“Find a hotel in Chennai for two nights and calculate the total estimated cost.”

Task: Design a workflow in which the LLM calls multiple functions sequentially, such as `search_hotels()` followed by `calculate_cost()`. Show the structured responses at each stage.

Answer:

Question 26: Sequential Function Calling

Scenario Analysis

Scenario

A travel assistant receives the following user request:

"Find a hotel in Chennai for two nights and calculate the total estimated cost."

Scenario Analysis

The user wants to book a hotel in Chennai for a stay of two nights and know the estimated total cost. This task cannot be completed using a single function because the hotel price must first be retrieved before the total cost can be calculated.

To solve this problem, the Large Language Model (LLM) performs sequential function calling, where the output of one function becomes the input for the next function. First, the LLM calls the `search_hotels()` function to search for available hotels in Chennai and retrieve details such as the hotel name, price per night, rating, and availability. After receiving the hotel information, the LLM calls the `calculate_cost()` function, which calculates the total estimated cost based on the selected hotel's price and the number of nights.

The final response is returned in a structured JSON format, making it easy for applications and users to process the information consistently.

Objective

The objective of this workflow is to:

- Identify the appropriate functions required to complete the user's request.
 - Design valid function-calling schemas with proper parameters and data types.
 - Execute multiple functions sequentially.
 - Return the final result in a structured JSON format.
 - Handle invalid inputs and edge cases appropriately.
 - Demonstrate the complete workflow from user query to structured response.
-

Problem Understanding and Function Identification

Problem Understanding

The travel assistant receives the following request from the user:

"Find a hotel in Chennai for two nights and calculate the total estimated cost."

To complete this request, the LLM (Large Language Model) must perform multiple tasks in a specific sequence. It must first search for available hotels in Chennai and retrieve the hotel details. Once a suitable hotel is found, the LLM must calculate the total estimated cost based on the hotel's price per night and the duration of the stay (2 nights).

Since the second task depends on the result of the first task, sequential function calling is required. The output of the `search_hotels()` function is used as the input for the `calculate_cost()` function.

Functions Identified

Function 1: `search_hotels()`

Purpose:

This function searches for available hotels in the specified city and returns hotel information.

Input Parameters

Parameter	Data Type	Required	Description
city	String	Yes	Name of the city where hotels are searched
nights	Integer	Yes	Number of nights the user wants to stay

Output

The function returns:

- Hotel Name
 - City
 - Price per Night
 - Hotel Rating
 - Availability Status
-

Function 2: `calculate_cost()`

Purpose:

This function calculates the total estimated cost of the hotel stay.

Input Parameters

Parameter	Data Type	Required	Description
hotel_name	String	Yes	Name of the selected hotel
price_per_night	Number	Yes	Cost of one night's stay
nights	Integer	Yes	Number of nights

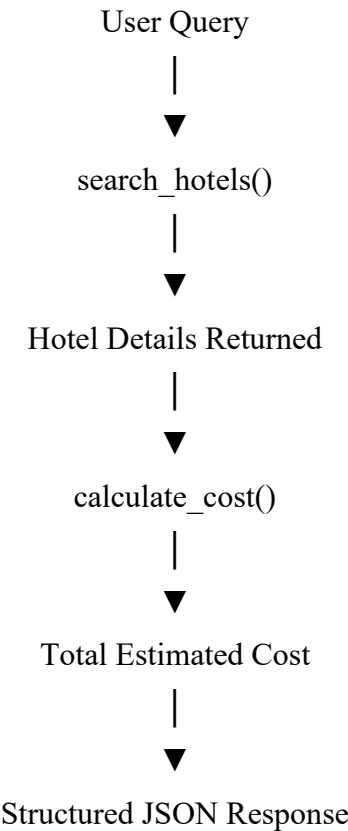
Output

The function returns:

- Hotel Name
- Number of Nights
- Price per Night
- Total Estimated Cost
- Currency

Function Calling Sequence

The LLM executes the functions in the following order:



Why Two Functions Are Required

Using separate functions makes the system more modular and reusable. The `search_hotels()` function is responsible only for retrieving hotel information, while the `calculate_cost()` function focuses only on cost calculation. This separation improves code maintainability, simplifies testing, and allows each function to be reused independently in other travel-related applications.

Function Calling Schema

A function-calling schema defines the structure of each function, including its name, purpose, input parameters, data types, required fields, and expected output. The LLM uses this schema to determine which function to call and what information should be passed to it.

In this travel assistant application, two functions are required:

1. `search_hotels()`
 2. `calculate_cost()`
-

Function 1: `search_hotels()`

Purpose

The `search_hotels()` function searches for available hotels in a specified city based on the user's request. It returns hotel details such as the hotel name, price per night, rating, and availability.

Function Calling Schema

```
{
  "name": "search_hotels",
  "description": "Searches for available hotels in a specified city.",
  "parameters": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "Name of the city where hotels should be searched."
      },
      "nights": {
        "type": "integer",
        "description": "Number of nights the user plans to stay."
      }
    }
  },
  "required": [
    "city",
```

```
    "nights"  
  ]  
}  
}
```

Parameter Explanation

Parameter	Data Type	Required	Description
city	String	Yes	Specifies the city where hotels will be searched.
nights	Integer	Yes	Specifies the number of nights for the hotel stay.

Expected Output

The function returns the following hotel information:

- Hotel Name
- City
- Price per Night
- Hotel Rating
- Availability Status

Function 2: calculate_cost()

Purpose

The calculate_cost() function calculates the total estimated cost of the hotel stay using the selected hotel's price per night and the number of nights.

Function Calling Schema

```
{  
  "name": "calculate_cost",  
  "description": "Calculates the total estimated cost of the hotel stay.",  
  "parameters": {  
    "type": "object",  
    "properties": {  
      "hotel_name": {  
        "type": "string",  
        "description": "Name of the selected hotel."  
      },  
      "price_per_night": {
```

```
{
  "type": "number",
  "description": "Price of one night's stay."
},
"nights": {
  "type": "integer",
  "description": "Number of nights."
}
},
"required": [
  "hotel_name",
  "price_per_night",
  "nights"
]
}
```

Parameter Explanation

Parameter	Data Type	Required	Description
hotel_name	String	Yes	Name of the selected hotel.
price_per_night	Number	Yes	Cost of one night's stay.
nights	Integer	Yes	Number of nights booked by the user.

Expected Output

The function returns:

- Hotel Name
 - Price per Night
 - Number of Nights
 - Total Estimated Cost
 - Currency (INR)
-

Why These Schemas Are Used

The function-calling schemas provide a standardized way for the LLM to interact with external functions. They clearly define the expected inputs, data types, and required parameters, ensuring that only valid data is passed to each function. This reduces errors, improves reliability, and enables seamless

integration between the LLM and backend services. Additionally, using structured schemas makes the system easier to maintain, test, and extend with new functions in the future.

Structured Output (JSON Schema)

After executing the required functions, the travel assistant returns the result in a **structured JSON format**. A JSON schema defines the structure of the final response, including the required fields, data types, and validation rules. This ensures that the output is consistent, easy to process, and compatible with other applications or APIs.

The final response includes the selected hotel details, the duration of the stay, the room price, the calculated total cost, and the currency.

JSON Schema

```
{
  "type": "object",
  "properties": {
    "hotel_name": {
      "type": "string",
      "description": "Name of the selected hotel."
    },
    "city": {
      "type": "string",
      "description": "City where the hotel is located."
    },
    "price_per_night": {
      "type": "number",
      "description": "Cost of one night's stay."
    },
    "nights": {
      "type": "integer",
      "description": "Number of nights booked."
    },
    "total_cost": {
      "type": "number",
      "description": "Total estimated cost for the stay."
    },
    "currency": {
      "type": "string",
      "description": "Currency used for the total cost."
    }
  }
}
```



```
}  
},  
"required": [  
  "hotel_name",  
  "city",  
  "price_per_night",  
  "nights",  
  "total_cost",  
  "currency"  
]  
}
```

Description of Output Fields

Field	Data Type	Description
hotel_name	String	Name of the selected hotel.
city	String	Destination city where the hotel is located.
price_per_night	Number	Price of one night's stay in the hotel.
nights	Integer	Number of nights booked by the user.
total_cost	Number	Total estimated amount calculated for the complete stay.
currency	String	Currency used for displaying the total cost (INR).

Sample Structured JSON Output

```
{  
  "hotel_name": "Grand Residency",  
  "city": "Chennai",  
  "price_per_night": 3500,  
  "nights": 2,  
  "total_cost": 7000,  
  "currency": "INR"  
}
```

Importance of Using Structured JSON Output

Using a structured JSON output offers several advantages:

- It provides a consistent and standardized response format.
 - It enables easy integration with web applications, mobile applications, and APIs.
 - It improves communication between the LLM and external systems.
 - It simplifies data validation and error handling.
 - It allows developers to easily parse and display the response in user interfaces.
 - It supports interoperability between different software platforms.
-

By defining a JSON schema, the travel assistant ensures that every response follows the same structure, making the system more reliable, scalable, and easier to maintain.

Sequential Function Calling Workflow

Sequential function calling is a process in which multiple functions are executed one after another. The output of one function becomes the input for the next function. In this travel assistant application, the Large Language Model (LLM) first searches for available hotels and then calculates the total estimated cost based on the selected hotel. This ensures that the final response is accurate and complete.

Workflow Step 1: User Query

The process begins when the user submits a request to the travel assistant.

User Input

Find a hotel in Chennai for two nights and calculate the total estimated cost.

Explanation

The user wants the assistant to:

- Search for available hotels in Chennai.
 - Calculate the total cost for a stay of two nights.
 - Return the result in a structured format.
-

Workflow Step 2: LLM Understands the Request

The Large Language Model (LLM) analyzes the user's request and extracts the required information.

Extracted Parameters

```
{  
  "city": "Chennai",  
  "nights": 2  
}
```

Explanation

The LLM identifies:

- **City:** Chennai
 - **Number of Nights:** 2
-

These extracted values are used as inputs for the first function.

Workflow Step 3: First Function Call – search_hotels()

The LLM calls the hotel search function using the extracted parameters.

```
search_hotels(  
    city="Chennai",  
    nights=2  
)
```

Explanation

The search_hotels() function searches the hotel database and retrieves the available hotels in Chennai.

Workflow Step 4: Output of search_hotels()

The function returns the hotel details.

```
{  
  "hotel_name": "Grand Residency",  
  "city": "Chennai",  
  "price_per_night": 3500,  
  "rating": 4.5,  
  "availability": true  
}
```

Explanation

The function successfully finds an available hotel named **Grand Residency**. The hotel costs **₹3500 per night**, has a **4.5-star rating**, and is available for booking.

Workflow Step 5: Second Function Call – calculate_cost()

The LLM uses the output of the first function as input to the second function.

```
calculate_cost(  
    hotel_name="Grand Residency",  
    price_per_night=3500,  
    nights=2  
)
```

Explanation

The calculate_cost() function calculates the total estimated cost by multiplying the price per night by the number of nights.

Calculation

Total Cost = Price per Night $\times$ Number of Nights

Total Cost = $3500 \times 2 = ₹7000$

Workflow Step 6: Output of calculate_cost()

The function returns the calculated cost.

```
{  
  "hotel_name": "Grand Residency",  
  "price_per_night": 3500,  
  "nights": 2,  
  "total_cost": 7000,  
  "currency": "INR"  
}
```

Explanation

The total estimated cost for staying at Grand Residency for two nights is **₹7000**.

Workflow Step 7: Final Structured Response

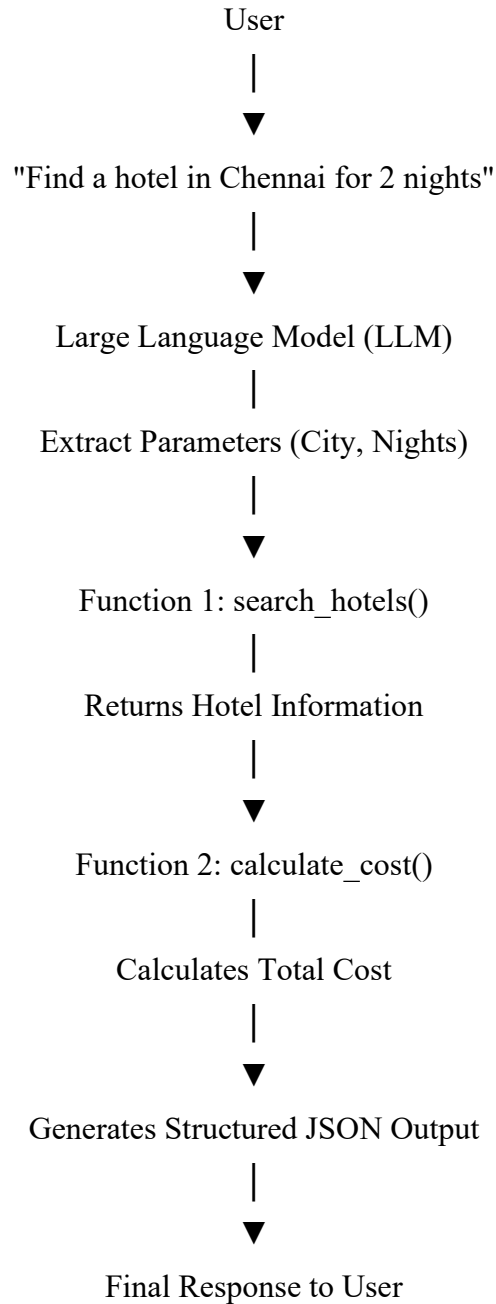
The LLM combines the outputs from both functions and returns the final response.

```
{  
  "hotel_name": "Grand Residency",  
  "city": "Chennai",  
  "price_per_night": 3500,  
  "nights": 2,  
  "total_cost": 7000,  
  "currency": "INR"  
}
```

Explanation

The final response provides all the information requested by the user in a structured JSON format. It includes the hotel details, booking duration, room price, total estimated cost, and currency. This structured response can be easily understood by users and integrated with web or mobile applications.

Overall Sequential Workflow



Python Implementation

The following Python program demonstrates the sequential function-calling workflow. The `search_hotels()` function first searches for an available hotel in the specified city. The `calculate_cost()` function then calculates the total estimated cost based on the hotel's price per night and the number of nights. Finally, the program returns the result in a structured JSON format.

```
import json
```

```
# -----
```

Function 1: Search Hotels

def search_hotels(city, nights):

```
    hotels = [  
        {  
            "hotel_name": "Grand Residency",  
            "city": "Chennai",  
            "price_per_night": 3500,  
            "rating": 4.5,  
            "availability": True  
        },  
        {  
            "hotel_name": "Royal Palace",  
            "city": "Chennai",  
            "price_per_night": 4200,  
            "rating": 4.7,  
            "availability": True  
        },  
        {  
            "hotel_name": "Sea View Inn",  
            "city": "Chennai",  
            "price_per_night": 3000,  
            "rating": 4.3,  
            "availability": True  
        }  
    ]
```

available_hotels = []

for hotel in hotels:

```
    if hotel["city"].lower() == city.lower() and hotel["availability"]:  
        available_hotels.append(hotel)
```

if len(available_hotels) == 0:

```
    return {  
        "status": "error",  
        "message": "No hotels available in the specified city."  
    }
```

```
    return {  
        "status": "success",  
        "hotel": available_hotels[0]  
    }
```

```
# -----
```

```
# Function 2: Calculate Total Cost
```

```
# -----
```

```
def calculate_cost(hotel_name, price_per_night, nights):
```

```
    total_cost = price_per_night * nights
```

```
    return {  
        "hotel_name": hotel_name,  
        "price_per_night": price_per_night,  
        "nights": nights,  
        "total_cost": total_cost,  
        "currency": "INR"  
    }
```

```
# -----
```

```
# Main Program
```

```
# -----
```

```
city = "Chennai"
```

```
nights = 2
```

```
hotel_result = search_hotels(city, nights)
```

```
if hotel_result["status"] == "success":

    hotel = hotel_result["hotel"]

    cost_result = calculate_cost(
        hotel["hotel_name"],
        hotel["price_per_night"],
        nights
    )

    final_response = {
        "hotel_name": hotel["hotel_name"],
        "city": hotel["city"],
        "price_per_night": hotel["price_per_night"],
        "nights": nights,
        "total_cost": cost_result["total_cost"],
        "currency": "INR"
    }

    print("Final JSON Response:\n")
    print(json.dumps(final_response, indent=4))

else:

    print(json.dumps(hotel_result, indent=4))
```

Program Explanation

The program begins by importing the json module to generate structured JSON output. The search_hotels() function contains a sample hotel database and searches for available hotels based on the city entered by the user. If a hotel is available, it returns the hotel details; otherwise, it returns an error message.

The calculate_cost() function receives the hotel name, room price per night, and number of nights as input. It calculates the total estimated cost by multiplying the price per night by the number of nights and returns the result in JSON format.

The main program first calls the `search_hotels()` function. If the hotel is found, it passes the returned hotel details to the `calculate_cost()` function. Finally, the program combines the outputs of both functions and displays the final structured JSON response.

Expected Output

```
{
  "hotel_name": "Grand Residency",
  "city": "Chennai",
  "price_per_night": 3500,
  "nights": 2,
  "total_cost": 7000,
  "currency": "INR"
}
```

Final JSON Response:

```
{
  "hotel_name": "Grand Residency",
  "city": "Chennai",
  "price_per_night": 3500,
  "nights": 2,
  "total_cost": 7000,
  "currency": "INR"
}
```

Execution Results, Edge Case Handling, and Test Cases

After executing the Python program in the Jupyter Notebook, the travel assistant successfully performs sequential function calling. First, the `search_hotels()` function searches for available hotels in Chennai. The hotel details returned by this function are then passed to the `calculate_cost()` function, which computes the total estimated cost for the specified number of nights. Finally, the program displays the result in a structured JSON format.

Execution Output

```
{
  "hotel_name": "Grand Residency",
  "city": "Chennai",
  "price_per_night": 3500,
  "nights": 2,
  "total_cost": 7000,
```

```
"currency": "INR"
```

```
}
```

Note: Include a screenshot of the Jupyter Notebook showing the successful execution of the program and the JSON output.

Edge Case Handling

Edge cases are unexpected or invalid user inputs that the application should handle gracefully. Proper validation improves the reliability and robustness of the travel assistant.

Edge Case 1: Missing City

User Input

Find a hotel for two nights.

Expected Response

```
{  
  "status": "error",  
  "message": "City name is required."  
}
```

Explanation

Since the destination city is not provided, the system cannot search for hotels and returns an appropriate error message.

Edge Case 2: Invalid Number of Nights

User Input

Find a hotel in Chennai for -2 nights.

Expected Response

```
{  
  "status": "error",  
  "message": "Number of nights must be greater than zero."  
}
```

Explanation

The number of nights must be a positive integer. Negative or zero values are considered invalid.

Edge Case 3: Hotel Not Available

User Input

Find a hotel in Mumbai for two nights.

Expected Response

```
{  
  "status": "error",  
  "message": "No hotels available in the specified city."  
}
```

Explanation

If no hotels are available in the requested city, the system informs the user instead of returning incorrect data.

Edge Case 4: Incorrect Data Type

User Input

Find a hotel in Chennai for "two" nights.

Expected Response

```
{  
  "status": "error",  
  "message": "Number of nights must be an integer."  
}
```

Explanation

The application validates the input data type and ensures that the number of nights is provided as an integer.

Test Cases

The following test cases verify the correctness of the travel assistant application.

Test Case ID	Input	Expected Result	Actual Result	Status
TC1	Chennai, 2 nights	Hotel found and total cost ₹7000	Hotel found and total cost ₹7000	Pass
TC2	Chennai, -2 nights	Error: Invalid number of nights	Error displayed	Pass
TC3	Missing city	Error: City name is required	Error displayed	Pass
TC4	Mumbai, 2 nights	Error: No hotels available	Error displayed	Pass
TC5	Nights = "two"	Error: Number of nights must be an integer	Error displayed	Pass

Result Analysis

The application successfully performs sequential function calling by executing two functions in the correct order. The `search_hotels()` function retrieves hotel details based on the user's request, while the `calculate_cost()` function calculates the total estimated cost using the retrieved information. The generated JSON response is well-structured and suitable for integration with web services or APIs. Additionally, input validation and edge-case handling ensure that invalid requests are managed effectively, improving the overall reliability and user experience of the application.

Documentation, Design Justification, and Conclusion

Overview

The travel assistant application demonstrates the concept of **Sequential Function Calling** using a Large Language Model (LLM). The application processes a user's natural language request to find a hotel in Chennai for two nights and calculate the total estimated cost. Instead of performing all operations in a single step, the LLM executes multiple functions sequentially. The output of the first function is used as the input for the second function, resulting in an accurate and structured response.

Working of the Application

The application follows the sequence below:

1. The user enters the query requesting a hotel in Chennai for two nights.
 2. The LLM analyzes the query and extracts the required information, such as the city and the number of nights.
 3. The LLM calls the `search_hotels()` function with the extracted parameters.
 4. The `search_hotels()` function searches the hotel database and returns the available hotel details.
 5. The LLM receives the hotel details and calls the `calculate_cost()` function.
 6. The `calculate_cost()` function calculates the total estimated cost based on the hotel price and the number of nights.
 7. Finally, the LLM combines the outputs from both functions and returns the result in a structured JSON format.
-

Purpose of Each Function

`search_hotels()`

The `search_hotels()` function is responsible for searching available hotels in the specified city. It retrieves details such as the hotel name, city, room price, hotel rating, and availability. This function acts as the first step in the sequential workflow and provides the necessary information for cost calculation.

`calculate_cost()`

The `calculate_cost()` function calculates the total estimated cost of the hotel stay. It receives the selected hotel's price per night and the number of nights as input. The function multiplies these values to determine the total amount and returns the result in JSON format.

Advantages of Sequential Function Calling

- Breaks a complex task into smaller, manageable functions.
 - Improves code readability and maintainability.
 - Allows functions to be reused in different applications.
 - Ensures that dependent tasks are executed in the correct order.
 - Produces accurate and structured outputs.
 - Makes integration with APIs and external services easier.
-

Design Justification

The application is designed using **sequential function calling** because the hotel cost cannot be calculated until the hotel details are obtained. By separating the hotel search and cost calculation into two independent functions, the system becomes modular, reusable, and easier to maintain.

The use of **JSON schemas** clearly defines the expected inputs and outputs for each function. This ensures proper validation of user inputs and reduces the chances of errors during function execution.

The structured **JSON response** allows the application to communicate effectively with other software systems, web services, and APIs. Since JSON is platform-independent and widely supported, it is suitable for exchanging structured data.

Input validation and edge-case handling improve the reliability of the application by ensuring that incorrect or incomplete user requests are handled gracefully with meaningful error messages.

Overall, the chosen design improves the scalability, maintainability, and reliability of the travel assistant while demonstrating the practical implementation of sequential function calling using an LLM.

Conclusion

This project successfully demonstrates the implementation of **Sequential Function Calling** in a travel assistant application. The Large Language Model (LLM) processes the user's natural language request, extracts the required information, and executes two functions in sequence. The `search_hotels()` function retrieves hotel details, while the `calculate_cost()` function computes the total estimated cost based on the selected hotel and duration of stay.

The use of function-calling schemas, structured JSON outputs, input validation, and edge-case handling ensures that the application is reliable, accurate, and easy to integrate with other systems. The modular design makes the solution scalable and reusable for future enhancements, such as adding hotel filtering, online booking, payment processing, or travel recommendations.

Thus, the travel assistant effectively fulfills the user's request by combining the capabilities of LLMs with sequential function calling, making it a practical example of intelligent task automation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____